

LAPORAN PRAKTIKUM PERTEMUAN 5
“Mengambil Data dari REST API (HTTP, Async, dan FutureBuilder)”



Disusun Oleh :
Adelia Rizky Cantika (20240801059)

Dosen Pengampu :
JEFRY SUNUPURWA ASRI , S.Kom., M.Kom

Mata Kuliah :
Pemrograman Mobile
CR002

FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
2026

Langkah Praktikum

Bagian A: SharedPreferences (Pengaturan Sederhana)

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Praktikum 5',
      home: const PengaturanPage(),
    );
  }
}

class PengaturanPage extends StatefulWidget {
  const PengaturanPage({super.key});

  @override
  State<PengaturanPage> createState() => _PengaturanPageState();
}

class _PengaturanPageState extends State<PengaturanPage> {
  final _controller = TextEditingController();

  bool _gelap = false;
  int _bukaKe = 0;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  // Memuat data yang tersimpan
```

```

Future<void> _muat() async {
  final prefs = await SharedPreferences.getInstance();

  if (!mounted) return;

  setState(() {
    _controller.text = prefs.getString('nama') ?? "";
    _gelap = prefs.getBool('gelap') ?? false;
    _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
  });

  await prefs.setInt('bukaKe', _bukaKe);
}

// Menyimpan nama
Future<void> _simpanNama() async {
  final prefs = await SharedPreferences.getInstance();

  await prefs.setString(
    'nama',
    _controller.text.trim(),
  );

  if (!mounted) return;

  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Nama disimpan'),
    ),
  );
}

// Mengubah dan menyimpan tema
Future<void> _ubahTema(bool nilai) async {
  setState(() {
    _gelap = nilai;
  });

  final prefs = await SharedPreferences.getInstance();

  await prefs.setBool('gelap', nilai);
}

@override
Widget build(BuildContext context) {
  return Theme(
    data: ThemeData(

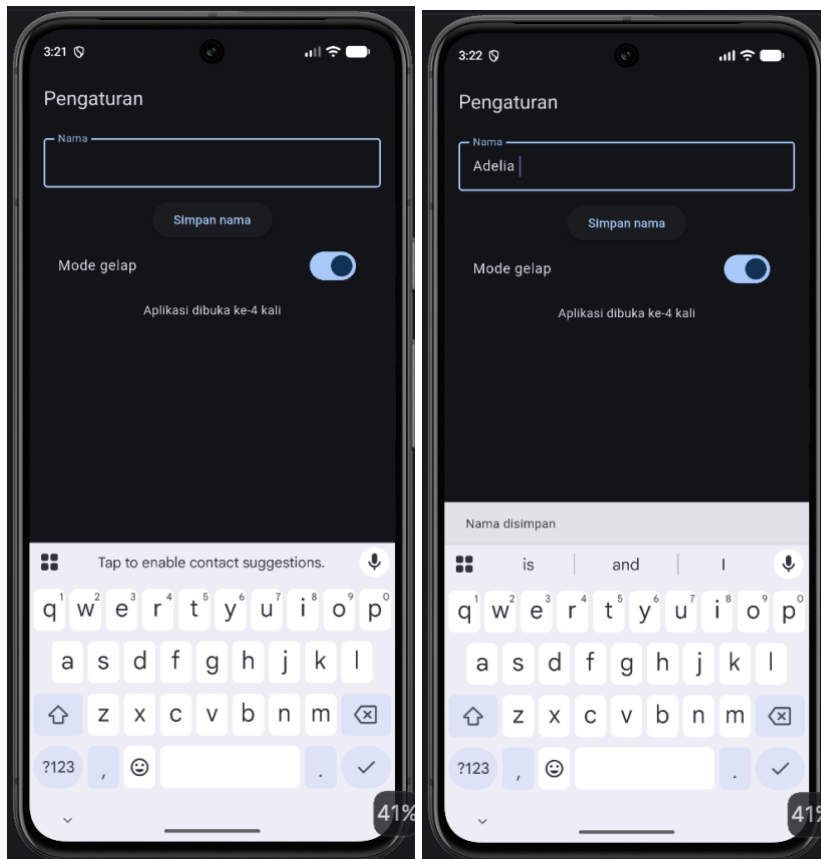
```

```

    brightness: _gelap ? Brightness.dark : Brightness.light,
    colorSchemeSeed: Colors.blue,
    useMaterial3: true,
  ),
  child: Scaffold(
    appBar: AppBar(
      title: const Text('Pengaturan'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          TextField(
            controller: _controller,
            decoration: const InputDecoration(
              labelText: 'Nama',
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: _simpanNama,
            child: const Text('Simpan nama'),
          ),
          SwitchListTile(
            title: const Text('Mode gelap'),
            value: _gelap,
            onChanged: _ubahTema,
          ),
          const SizedBox(height: 12),
          Text('Aplikasi dibuka ke-$_bukaKe kali'),
        ],
      ),
    ),
  );
}

```

Tampilan:



Bagian B: Menyiapkan SQLite

```
import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
        title: 'Praktikum 5',
        debugShowCheckedModeBanner: false,
        home: const PengaturanPage(),
    );
}
}
```

```
// =====
// MODEL CATATAN
// =====
```

```
class Catatan {
    final int? id;
    final String judul;
    final String isi;

    const Catatan({
        this.id,
        required this.judul,
        required this.isi,
    });
```

```
    Map<String, Object?> toMap() {
        return {
            'id': id,
            'judul': judul,
            'isi': isi,
        };
    }
}
```

```
factory Catatan.fromMap(Map<String, Object?> m) {
```

```

        return Catatan(
            id: m['id'] as int,
            judul: m['judul'] as String,
            isi: m['isi'] as String,
        );
    }
}

// =====

// DATABASE HELPER

// =====

class DbHelper {
    static Database? _db;

    static Future<Database> get database async {
        if (_db != null) {
            return _db!;
        }

        final path = p.join(
            await getDatabasesPath(),
            'catatan.db',
        );

        _db = await openDatabase(
            path,
            version: 1,
            onCreate: (db, version) {
                return db.execute(
                    'CREATE TABLE catatan('

```

```
        'id INTEGER PRIMARY KEY AUTOINCREMENT, '  
        'judul TEXT NOT NULL, '  
        'isi TEXT NOT NULL'  
    '),  
    );  
},  
);
```

```
    return _db!;  
}
```

```
static Future<int> tambah(Catatan c) async {  
    final db = await database;  
  
    return db.insert(  
        'catatan',  
        c.toMap(),  
    );  
}
```

```
static Future<List<Catatan>> semua() async {  
    final db = await database;  
  
    final rows = await db.query(  
        'catatan',  
        orderBy: 'id DESC',  
    );  
  
    return rows.map(Catatan.fromMap).toList();  
}
```



```

static Future<int> ubah(Catatan c) async {
    final db = await database;

    return db.update(
        'catatan',
        c.toMap(),
        where: 'id = ?',
        whereArgs: [c.id],
    );
}

```

```

static Future<int> hapus(int id) async {
    final db = await database;

    return db.delete(
        'catatan',
        where: 'id = ?',
        whereArgs: [id],
    );
}
}

```

```

// =====
// HALAMAN UTAMA
// =====

```

```

class PengaturanPage extends StatelessWidget {
    const PengaturanPage({super.key});

    @override
    Widget build(BuildContext context) {

```

```
return Scaffold(  
  appBar: AppBar(  
    title: const Text('Praktikum 5'),  
  ),  
  body: const Center(  
    child: Text(  
      'Database SQLite berhasil disiapkan',  
      style: TextStyle(  
        fontSize: 18,  
      ),  
    ),  
  ),  
);  
}
```

Tampilan:



Bagian C: Halaman Daftar Catatan (Read dan Delete)

```
class CatatanPage extends StatefulWidget {  
  const CatatanPage({super.key});  
  
  @override  
  State<CatatanPage> createState() => _CatatanPageState();  
}
```

```
class _CatatanPageState extends State<CatatanPage> {  
  late Future<List<Catatan>>> _future;  
  
  @override  
  void initState() {  
    super.initState();  
    _future = DbHelper.semua();  
  }
```

```
  void _muat() {  
    setState(() {  
      _future = DbHelper.semua();  
    });  
  }
```

```
  Future<void> _buka([Catatan? catatan]) async {  
    await Navigator.push(  
      context,  
      MaterialPageRoute(  
        builder: (_) => FormCatatanPage(  
          catatan: catatan,  
        ),  
      ),  
    );  
  }
```

```
);
```

```
if (!mounted) return;
```

```
  _muat();
```

```
}
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Scaffold(
```

```
    appBar: AppBar(
```

```
      title: const Text('Catatan Saya'),
```

```
    ),
```

```
    body: FutureBuilder<List<Catatan>>(</pre></div>
```

```

return const Center(
  child: Text('Belum ada catatan'),
);
}

return ListView.builder(
  itemCount: data.length,
  itemBuilder: (context, i) {
    final c = data[i];

    return ListTile(
      title: Text(c.judul),
      subtitle: Text(
        c.isi,
        maxLines: 2,
        overflow: TextOverflow.ellipsis,
      ),
      onTap: () => _buka(c),
      trailing: IconButton(
        icon: const Icon(Icons.delete),
        onPressed: () async {
          await DbHelper.hapus(c.id!);
          _muat();
        },
      ),
    );
  },
);

floatingActionButton: FloatingActionButton(

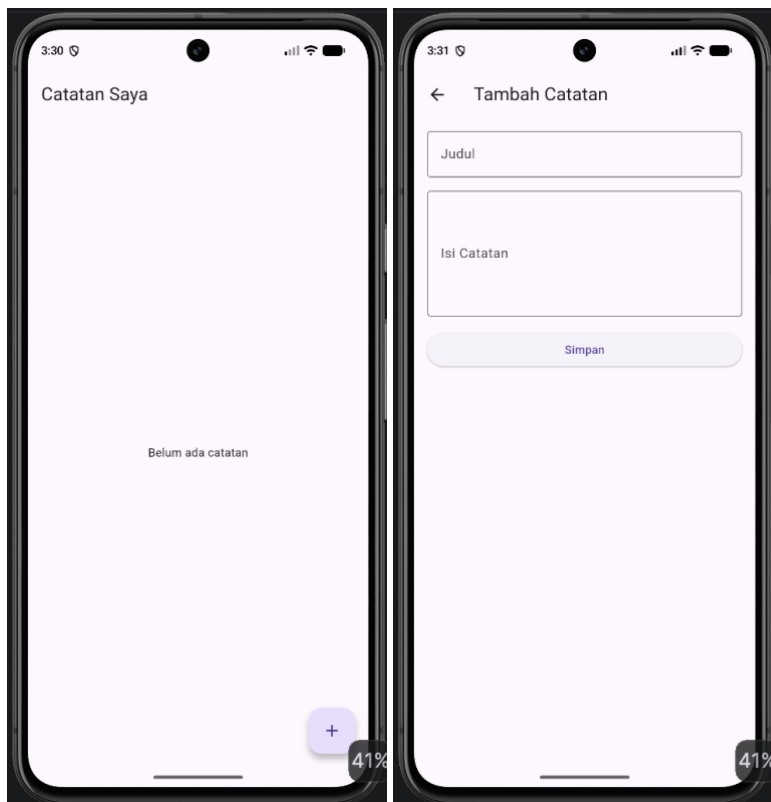
```

```

        onPressed: () => _buka(),
        child: const Icon(Icons.add),
      ),
    );
  }
}

```

Tampilan:



Bagian D: Form Tambah dan Ubah Catatan (Create dan Update)

```

class FormCatatanPage extends StatefulWidget {
  final Catatan? catatan;

  const FormCatatanPage({super.key, this.catatan});

  @override
  State<FormCatatanPage> createState() => _FormCatatanPageState();
}

class _FormCatatanPageState extends State<FormCatatanPage> {
  final _formKey = GlobalKey<FormState>();

```

```
late final TextEditingController _judul;
```

```
late final TextEditingController _isi;
```

```
@override
```

```
void initState() {
```

```
    super.initState();
```

```
    _judul = TextEditingController(text: widget.catatan?.judul ?? "");
```

```
    _isi = TextEditingController(text: widget.catatan?.isi ?? "");
```

```
}
```

```
@override
```

```
void dispose() {
```

```
    _judul.dispose();
```

```
    _isi.dispose();
```

```
    super.dispose();
```

```
}
```

```
Future<void> _simpan() async {
```

```
    if (!_formKey.currentState!.validate()) return;
```

```
    final c = Catatan(
```

```
        id: widget.catatan?.id,
```

```
        judul: _judul.text.trim(),
```

```
        isi: _isi.text.trim(),
```

```
    );
```

```
    if (widget.catatan == null) {
```

```
        await DbHelper.tambah(c);
```

```
    } else {
```

```
        await DbHelper.ubah(c);
```

```
    }
```

```
if (!mounted) return;  
Navigator.pop(context);  
}
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  final baru = widget.catatan == null;
```

```
  return Scaffold(  
    appBar: AppBar(  
      title: Text(baru ? 'Catatan Baru' : 'Ubah Catatan'),  
    ),  
    body: Form(  
      key: _formKey,  
      child: ListView(  
        padding: const EdgeInsets.all(16),  
        children: [  
          TextFormField(  
            controller: _judul,  
            decoration: const InputDecoration(  
              labelText: 'Judul',  
              border: OutlineInputBorder(),  
            ),  
            validator: (v) =>  
              (v == null || v.trim().isEmpty)  
                ? 'Judul wajib diisi'  
                : null,  
          ),  
          const SizedBox(height: 12),  
          TextFormField(  

```



```
    controller: _isi,
    maxLines: 6,
    decoration: const InputDecoration(
      labelText: 'Isi catatan',
      border: OutlineInputBorder(),
    ),
    validator: (v) =>
      (v == null || v.trim().isEmpty)
        ? 'Isi wajib diisi'
        : null,
  ),
  const SizedBox(height: 16),
  ElevatedButton(
    onPressed: _simpan,
    child: const Text('Simpan'),
  ),
],
),
),
);
}
```

Tampilan:



Latihan Mandiri:

```
import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';
import 'package:shared_preferences/shared_preferences.dart';
```

```
void main() {
  runApp(const MyApp());
}
```

```
// =====
// MODEL
// =====
```

```
class Catatan {
  final int? id;
  final String judul;
  final String isi;
  final String dibuat;

  const Catatan({
    this.id,
    required this.judul,
    required this.isi,
```

```
        required this.dibuat,
    });
```

```
Map<String, Object?> toMap() {
    return {
        'id': id,
        'judul': judul,
        'isi': isi,
        'dibuat': dibuat,
    };
}
```

```
factory Catatan.fromMap(Map<String, Object?> m) {
    return Catatan(
        id: m['id'] as int,
        judul: m['judul'] as String,
        isi: m['isi'] as String,
        dibuat: (m['dibuat'] as String?) ?? "",
    );
}
}
```

```
// =====
// DATABASE HELPER
// =====
```

```
class DbHelper {
    static Database? _db;

    static Future<Database> get database async {
        if (_db != null) return _db!;

        final path = p.join(
            await getDatabasesPath(),
            'catatan.db',
        );

        _db = await openDatabase(
            path,
            version: 2,

            // Membuat database baru
            onCreate: (db, version) {
                return db.execute(
                    'CREATE TABLE catatan('
                    'id INTEGER PRIMARY KEY AUTOINCREMENT, '
                    'judul TEXT NOT NULL, '
```

```

        'isi TEXT NOT NULL, '
        'dibuat TEXT NOT NULL'
    '),
);
},

// Upgrade database dari versi sebelumnya
onUpgrade: (db, oldVersion, newVersion) async {
    if (oldVersion < 2) {
        await db.execute(
            'ALTER TABLE catatan ADD COLUMN dibuat TEXT',
        );

        // Memberikan tanggal pada data lama
        await db.update(
            'catatan',
            {
                'dibuat': DateTime.now().toIso8601String(),
            },
            where: 'dibuat IS NULL',
        );
    }
},

return _db!;
}

// CREATE
static Future<int> tambah(Catatan c) async {
    final db = await database;

    return db.insert(
        'catatan',
        c.toMap(),
    );
}

// READ
static Future<List<Catatan>> semua({
    String orderBy = 'id DESC',
}) async {
    final db = await database;

    final rows = await db.query(
        'catatan',
        orderBy: orderBy,
    );
}

```

```

    );

    return rows.map(Catatan.fromMap).toList();
}

// SEARCH
static Future<List<Catatan>> cari(
    String kata, {
        String orderBy = 'id DESC',
    }) async {
    final db = await database;

    final rows = await db.query(
        'catatan',
        where: 'judul LIKE ?',
        whereArgs: ['%$kata%'],
        orderBy: orderBy,
    );

    return rows.map(Catatan.fromMap).toList();
}

// UPDATE
static Future<int> ubah(Catatan c) async {
    final db = await database;

    return db.update(
        'catatan',
        c.toMap(),
        where: 'id = ?',
        whereArgs: [c.id],
    );
}

// DELETE
static Future<int> hapus(int id) async {
    final db = await database;

    return db.delete(
        'catatan',
        where: 'id = ?',
        whereArgs: [id],
    );
}

// =====

```

```
// MY APP
```

```
// =====
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      title: 'Catatan SQLite',  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(  
          seedColor: Colors.blue,  
        ),  
        useMaterial3: true,  
      ),  
      home: const CatatanPage(),  
    );  
  }  
}
```

```
// =====
```

```
// HALAMAN DAFTAR CATATAN
```

```
// =====
```

```
class CatatanPage extends StatefulWidget {  
  const CatatanPage({super.key});  
  
  @override  
  State<CatatanPage> createState() => _CatatanPageState();  
}
```

```
class _CatatanPageState extends State<CatatanPage> {  
  late Future<List<Catatan>>> _future;
```

```
  final TextEditingController _searchController =  
    TextEditingController();
```

```
  // Menyimpan pilihan urutan  
  String _urutan = 'terbaru';
```

```
  @override  
  void initState() {  
    super.initState();
```

```
    _future = DbHelper.semua();
```

```

        _searchController.addListener(_cari);

        // Membaca pilihan urutan yang tersimpan
        _ambilUrutan();
    }

    @override
    void dispose() {
        _searchController.removeListener(_cari);
        _searchController.dispose();
        super.dispose();
    }

    // =====
    // MENGAMBIL PILIHAN URUTAN DARI SHARED PREFERENCES
    // =====

    Future<void> _ambilUrutan() async {
        final prefs = await SharedPreferences.getInstance();

        final tersimpan = prefs.getString('urutan') ?? 'terbaru';

        setState(() {
            _urutan = tersimpan;
        });

        _muat();
    }

    // =====
    // MENYIMPAN PILIHAN URUTAN
    // =====

    Future<void> _simpanUrutan(String nilai) async {
        final prefs = await SharedPreferences.getInstance();

        await prefs.setString(
            'urutan',
            nilai,
        );

        setState(() {
            _urutan = nilai;
        });

        _muat();
    }

```

```

}

// =====
// MENENTUKAN ORDER BY
// =====

String get _orderBy {
    if (_urutan == 'terlama') {
        return 'id ASC';
    }

    return 'id DESC';
}

// =====
// LOAD DATA
// =====

void _muat() {
    final kata = _searchController.text.trim();

    setState(() {
        if (kata.isEmpty) {
            _future = DbHelper.semua(
                orderBy: _orderBy,
            );
        } else {
            _future = DbHelper.cari(
                kata,
                orderBy: _orderBy,
            );
        }
    });
}

// =====
// SEARCH
// =====

void _cari() {
    final kata = _searchController.text.trim();

    setState(() {
        if (kata.isEmpty) {
            _future = DbHelper.semua(
                orderBy: _orderBy,
            );
        }
    });
}

```



```

    } else {
        _future = DbHelper.cari(
            kata,
            orderBy: _orderBy,
        );
    }
});
}

// =====
// KONFIRMASI HAPUS
// =====

Future<void> _konfirmasiHapus(Catatan catatan) async {
    final hasil = await showDialog<bool>(
        context: context,
        builder: (context) {
            return AlertDialog(
                title: const Text('Konfirmasi'),
                content: const Text(
                    'Hapus catatan ini?',
                ),
                actions: [
                    TextButton(
                        onPressed: () {
                            Navigator.pop(context, false);
                        },
                        child: const Text('Batal'),
                    ),
                    ElevatedButton(
                        onPressed: () {
                            Navigator.pop(context, true);
                        },
                        child: const Text('Hapus'),
                    ),
                ],
            );
        },
    );

    if (hasil == true) {
        await DbHelper.hapus(catatan.id!);
        _muat();
    }
}

// =====

```

```
// BUKA FORM
```

```
// =====
```

```
Future<void> _buka([Catatan? catatan]) async {  
  await Navigator.push(  
    context,  
    MaterialPageRoute(  
      builder: (_) => FormCatatanPage(  
        catatan: catatan,  
      ),  
    ),  
  );  
}  
  
_muat();  
}
```

```
// =====
```

```
// BUILD
```

```
// =====
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('Daftar Catatan'),  
  
    actions: [  
      PopupMenuButton<String>(  
        icon: const Icon(Icons.sort),  
  
        initialValue: _urutan,  
  
        onSelect: (value) {  
          _simpanUrutan(value);  
        },  
  
        itemBuilder: (context) => const [  
          PopupMenuItem(  
            value: 'terbaru',  
            child: Text('Terbaru'),  
          ),  
          PopupMenuItem(  
            value: 'terlama',  
            child: Text('Terlama'),  
          ),  
        ],  
      ),  
    ],  
  ),
```

```
],  
)
```

```
body: Column(  
  children: [
```

```
    // =====  
    // SEARCH  
    // =====
```

```
    Padding(  
      padding: const EdgeInsets.all(16),  
      child: TextField(  
        controller: _searchController,  
        decoration: InputDecoration(  
          labelText: 'Cari catatan',  
          hintText: 'Masukkan judul catatan',  
          prefixIcon: const Icon(Icons.search),
```

```
          suffixIcon:  
            _searchController.text.isNotEmpty  
              ? IconButton(  
                icon: const Icon(Icons.clear),  
                onPressed: () {  
                  _searchController.clear();  
                },  
              )  
            : null,
```

```
        border: const OutlineInputBorder(),  
      ),  
    ),  
  ),
```

```
  // =====  
  // INFORMASI URUTAN  
  // =====
```

```
  Padding(  
    padding: const EdgeInsets.symmetric(  
      horizontal: 16,  
    ),  
    child: Align(  
      alignment: Alignment.centerLeft,  
      child: Text(  
        'Urutan: ${_urutan == 'terbaru' ? 'Terbaru' : 'Terlama'}',  
        style: const TextStyle(  
          fontWeight: FontWeight.w500,
```

```

    ),
    ),
    ),
),

const SizedBox(height: 8),

// =====
// LIST CATATAN
// =====

Expanded(
  child: FutureBuilder<List<Catatan>>(
    future: _future,

    builder: (context, snapshot) {
      if (snapshot.connectionState ==
        ConnectionState.waiting) {
        return const Center(
          child: CircularProgressIndicator(),
        );
      }

      if (snapshot.hasError) {
        return Center(
          child: Text(
            'Terjadi kesalahan: ${snapshot.error}',
          ),
        );
      }

      final data = snapshot.data ?? [];

      if (data.isEmpty) {
        return const Center(
          child: Text(
            'Belum ada catatan',
          ),
        );
      }

      return ListView.builder(
        itemCount: data.length,

        itemBuilder: (context, index) {
          final c = data[index];

```

```

return Card(
  margin: const EdgeInsets.symmetric(
    horizontal: 16,
    vertical: 6,
  ),

  child: ListTile(
    title: Text(
      c.judul,
      style: const TextStyle(
        fontWeight: FontWeight.bold,
      ),
    ),

    subtitle: Column(
      crossAxisAlignment:
        CrossAxisAlignment.start,
      children: [
        const SizedBox(height: 4),

        Text(
          c.isi,
          maxLines: 2,
          overflow:
            TextOverflow.ellipsis,
        ),

        const SizedBox(height: 6),

        Text(
          'Dibuat: ${c.dibuat}',
          style: const TextStyle(
            fontSize: 12,
          ),
        ),
      ],
    ),

    onTap: () {
      _buka(c);
    },

    trailing: IconButton(
      icon: const Icon(
        Icons.delete,
        color: Colors.red,
      ),

```

```

        onPressed: () {
          _konfirmasiHapus(c);
        },
      ),
    ),
  );
},
);
},
),
),
],
),

// =====
// TAMBAH CATATAN
// =====

floatingActionButton: FloatingActionButton(
  onPressed: () {
    _buka();
  },
  child: const Icon(Icons.add),
),
);
}
}

// =====
// FORM CATATAN
// =====

class FormCatatanPage extends StatefulWidget {
  final Catatan? catatan;

  const FormCatatanPage({
    super.key,
    this.catatan,
  });

  @override
  State<FormCatatanPage> createState() =>
    _FormCatatanPageState();
}

class _FormCatatanPageState

```

```

    extends State<FormCatatanPage> {
final _formKey = GlobalKey<FormState>();

late final TextEditingController _judul;
late final TextEditingController _isi;

@override
void initState() {
    super.initState();

    _judul = TextEditingController(
        text: widget.catatan?.judul ?? "",
    );

    _isi = TextEditingController(
        text: widget.catatan?.isi ?? "",
    );
}

@override
void dispose() {
    _judul.dispose();
    _isi.dispose();
    super.dispose();
}

// =====
// SIMPAN
// =====

Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) {
        return;
    }

    final sekarang =
        DateTime.now().toIso8601String();

    final c = Catatan(
        id: widget.catatan?.id,
        judul: _judul.text.trim(),
        isi: _isi.text.trim(),

        // Jika edit, tanggal dibuat tetap menggunakan
        // tanggal lama.
        dibuat:
            widget.catatan?.dibuat ?? sekarang,

```

```

);

// Tambah data baru
if (widget.catatan == null) {
  await DbHelper.tambah(c);
}

// Update data lama
else {
  await DbHelper.ubah(c);
}

if (!mounted) return;

Navigator.pop(context);
}

// =====
// BUILD
// =====

@override
Widget build(BuildContext context) {
  final baru = widget.catatan == null;

  return Scaffold(
    appBar: AppBar(
      title: Text(
        baru ? 'Catatan Baru' : 'Ubah Catatan',
      ),
    ),

    body: Form(
      key: _formKey,

      child: ListView(
        padding: const EdgeInsets.all(16),

        children: [
          // =====
          // JUDUL
          // =====

          TextFormField(
            controller: _judul,

            decoration: const InputDecoration(

```



```

        labelText: 'Judul',
        border: OutlineInputBorder(),
    ),

    validator: (v) {
      if (v == null ||
        v.trim().isEmpty) {
        return 'Judul wajib diisi';
      }

      return null;
    },
  ),

  const SizedBox(height: 12),

  // =====
  // ISI
  // =====

  TextFormField(
    controller: _isi,

    maxLines: 6,

    decoration: const InputDecoration(
      labelText: 'Isi catatan',
      border: OutlineInputBorder(),
    ),

    validator: (v) {
      if (v == null ||
        v.trim().isEmpty) {
        return 'Isi wajib diisi';
      }

      return null;
    },
  ),

  const SizedBox(height: 16),

  // =====
  // TOMBOL SIMPAN
  // =====

  ElevatedButton(

```

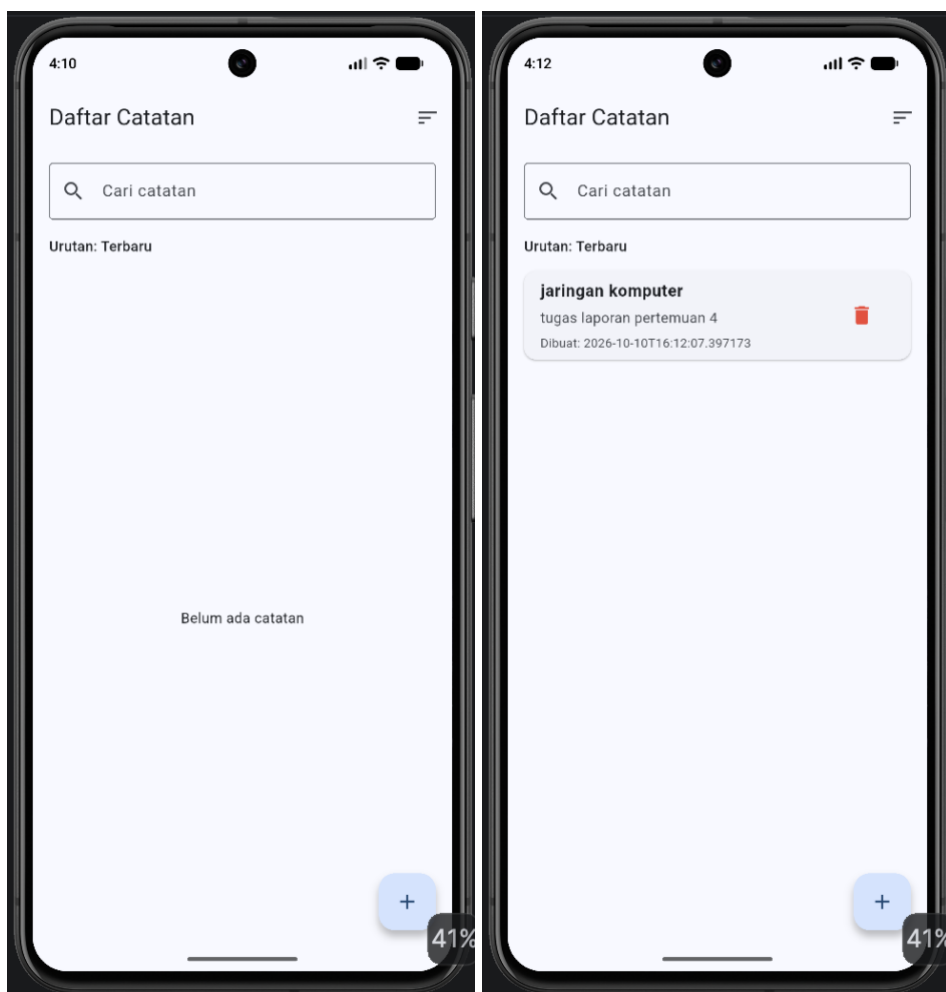
```

        onPressed: _simpan,

        child: const Text(
          'Simpan',
        ),
      ),
    ],
  ),
),
);
}
}

```

Tampilan:



Tugas Mandiri

```

import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

void main() {

```

```

runApp(const MyApp());
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  bool _gelap = false;
  bool _memuat = true;

  @override
  void initState() {
    super.initState();
    _muatPengaturan();
  }

  Future<void> _muatPengaturan() async {
    final prefs = await SharedPreferences.getInstance();

    if (!mounted) return;

    setState(() {
      _gelap = prefs.getBool('gelap') ?? false;
      _memuat = false;
    });
  }

  Future<void> _ubahTema(bool nilai) async {
    final prefs = await SharedPreferences.getInstance();

    await prefs.setBool('gelap', nilai);

    if (!mounted) return;

    setState(() {
      _gelap = nilai;
    });
  }

  @override
  Widget build(BuildContext context) {
    if (_memuat) {
      return const MaterialApp(

```

```

        home: Scaffold(
          body: Center(
            child: CircularProgressIndicator(),
          ),
        ),
      );
    }

    return MaterialApp(
      title: 'Pencatat Pengeluaran',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        brightness:
          _gelap ? Brightness.dark : Brightness.light,
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: PengeluaranPage(
        gelap: _gelap,
        onTemaChanged: _ubahTema,
      ),
    );
  }
}

```

```

class Pengeluaran {
  final int? id;
  final String nama;
  final int jumlah;
  final String kategori;
  final String tanggal;

  const Pengeluaran({
    this.id,
    required this.nama,
    required this.jumlah,
    required this.kategori,
    required this.tanggal,
  });
}

```

```

Map<String, Object?> toMap() {
  return {
    'id': id,
    'nama': nama,
    'jumlah': jumlah,
    'kategori': kategori,
    'tanggal': tanggal,
  };
}

```

```

    };
}

factory Pengeluaran.fromMap(
    Map<String, Object?> map,
) {
    return Pengeluaran(
        id: map['id'] as int,
        nama: map['nama'] as String,
        jumlah: map['jumlah'] as int,
        kategori: map['kategori'] as String,
        tanggal: map['tanggal'] as String,
    );
}

}

class DbHelper {
    static Database? _db;

    static Future<Database> get database async {
        if (_db != null) {
            return _db!;
        }

        final path = p.join(
            await getDatabasesPath(),
            'pengeluaran.db',
        );

        _db = await openDatabase(
            path,
            version: 1,
            onCreate: (db, version) async {
                await db.execute(
                    'CREATE TABLE pengeluaran('
                    'id INTEGER PRIMARY KEY AUTOINCREMENT, '
                    'nama TEXT NOT NULL, '
                    'jumlah INTEGER NOT NULL, '
                    'kategori TEXT NOT NULL, '
                    'tanggal TEXT NOT NULL)',
                );
            },
        );

        return _db!;
    }
}

```

```

static Future<int> tambah(
    Pengeluaran pengeluaran,
) async {
    final db = await database;

    return db.insert(
        'pengeluaran',
        pengeluaran.toMap(),
    );
}

static Future<List<Pengeluaran>> semua() async {
    final db = await database;

    final rows = await db.query(
        'pengeluaran',
        orderBy: 'tanggal DESC, id DESC',
    );

    return rows.map(Pengeluaran.fromMap).toList();
}

static Future<int> ubah(
    Pengeluaran pengeluaran,
) async {
    final db = await database;

    return db.update(
        'pengeluaran',
        pengeluaran.toMap(),
        where: 'id = ?',
        whereArgs: [pengeluaran.id],
    );
}

static Future<int> hapus(int id) async {
    final db = await database;

    return db.delete(
        'pengeluaran',
        where: 'id = ?',
        whereArgs: [id],
    );
}

static Future<int> total() async {
    final db = await database;

```

```

    final result = await db.rawQuery(
      'SELECT SUM(jumlah) AS total FROM pengeluaran',
    );

    return (result.first['total'] as int?) ?? 0;
  }
}

```

```

class PengeluaranPage extends StatefulWidget {
  final bool gelap;
  final Future<void> Function(bool) onTemaChanged;

```

```

  const PengeluaranPage({
    super.key,
    required this.gelap,
    required this.onTemaChanged,
  });

```

```

  @override
  State<PengeluaranPage> createState() =>
    _PengeluaranPageState();
}

```

```

class _PengeluaranPageState
  extends State<PengeluaranPage> {
  late Future<List<Pengeluaran>> _future;
  int _total = 0;

```

```

  @override
  void initState() {
    super.initState();
    _muatData();
  }

```

```

  Future<void> _muatData() async {
    final futureData = DbHelper.semua();
    final total = await DbHelper.total();

```

```

    if (!mounted) return;

```

```

    setState(() {
      _future = futureData;
      _total = total;
    });
  }
}

```

```
String _formatRupiah(int angka) {
  final teks = angka.toString();
  final buffer = StringBuffer();

  for (int i = 0; i < teks.length; i++) {
    if (i > 0 &&
        (teks.length - i) % 3 == 0) {
      buffer.write('.');
    }

    buffer.write(teks[i]);
  }

  return 'Rp $buffer';
}
```

```
Future<void> _bukaForm([
  Pengeluaran? pengeluaran,
]) async {
  await Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => FormPengeluaranPage(
        pengeluaran: pengeluaran,
      ),
    ),
  );

  if (!mounted) return;

  _muatData();
}
```

```
Future<void> _hapus(
  Pengeluaran pengeluaran,
) async {
  final hasil = await showDialog<bool>(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: const Text('Hapus Pengeluaran'),
        content: Text(
          'Hapus "${pengeluaran.nama}"?',
        ),
        actions: [
          TextButton(
            onPressed: () {
```



```

        Navigator.pop(context, false);
      },
      child: const Text('Batal'),
    ),
    ElevatedButton(
      onPressed: () {
        Navigator.pop(context, true);
      },
      child: const Text('Hapus'),
    ),
  ],
);
},
);

if (hasil == true) {
  await DbHelper.hapus(
    pengeluaran.id!,
  );

  if (!mounted) return;

  _muatData();

  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text(
        'Pengeluaran berhasil dihapus',
      ),
    ),
  );
}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Pencatat Pengeluaran'),
      actions: [
        Switch(
          value: widget.gelap,
          onChanged: widget.onTemaChanged,
        ),
      ],
    ),
    body: Column(

```

```
children: [
    Container(
      width: double.infinity,
      margin: const EdgeInsets.all(16),
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(
        color: Theme.of(context)
          .colorScheme
          .primaryContainer,
        borderRadius: BorderRadius.circular(16),
      ),
      child: Column(
        children: [
          const Text(
            'Total Pengeluaran',
            style: TextStyle(
              fontSize: 16,
            ),
          ),
          const SizedBox(height: 8),
          Text(
            _formatRupiah(_total),
            style: const TextStyle(
              fontSize: 28,
              fontWeight: FontWeight.bold,
            ),
          ),
        ],
      ),
    ),
  ],
),
Expanded(
  child: FutureBuilder<List<Pengeluaran>>(
    future: _future,
    builder: (context, snapshot) {
      if (snapshot.connectionState ==
        ConnectionState.waiting) {
        return const Center(
          child: CircularProgressIndicator(),
        );
      }

      if (snapshot.hasError) {
        return Center(
          child: Text(
            'Galat: ${snapshot.error}',
          ),
        );
      }
    },
  ),
)
```

```
}
```

```
final data = snapshot.data ?? [];
```

```
if (data.isEmpty) {  
  return const Center(  
    child: Text(  
      'Belum ada pengeluaran',  
    ),  
  );  
}
```

```
return ListView.builder(  
  padding: const EdgeInsets.only(  
    bottom: 80,  
  ),  
  itemCount: data.length,  
  itemBuilder: (context, index) {  
    final pengeluaran = data[index];
```

```
    return ListTile(  
      leading: CircleAvatar(  
        child: Text(  
          pengeluaran.nama[0]  
            .toUpperCase(),  
        ),  
      ),  
      title: Text(  
        pengeluaran.nama,  
        style: const TextStyle(  
          fontWeight: FontWeight.bold,  
        ),  
      ),  
      subtitle: Text(  
        '${pengeluaran.kategori} • '  
          '${pengeluaran.tanggal}',  
      ),  
      trailing: Row(  
        mainAxisAlignment: MainAxisAlignment.min,  
        children: [  
          Text(  
            _formatRupiah(  
              pengeluaran.jumlah,  
            ),  
            style: const TextStyle(  
              fontWeight: FontWeight.bold,  
            ),  
          ],  
        ),  
      ),  
    );  
  }  
);
```

```

    ),
    PopupMenuButton<String>(
      onSelected: (value) {
        if (value == 'ubah') {
          _bukaForm(
            pengeluaran,
          );
        } else if (value ==
          'hapus') {
          _hapus(
            pengeluaran,
          );
        }
      },
      itemBuilder: (context) {
        return const [
          PopupMenuItem(
            value: 'ubah',
            child: Text('Ubah'),
          ),
          PopupMenuItem(
            value: 'hapus',
            child: Text('Hapus'),
          ),
        ];
      },
    ),
  ],
),
onTap: () {
  _bukaForm(
    pengeluaran,
  );
},
);
},
);
},
),
),
],
),
floatingActionButton:
FloatingActionButton.extended(
  onPressed: () {
    _bukaForm();
  },

```

```

        icon: const Icon(Icons.add),
        label: const Text('Tambah'),
    ),
);
}
}

```

```

class FormPengeluaranPage extends StatefulWidget {
  final Pengeluaran? pengeluaran;

```

```

  const FormPengeluaranPage({
    super.key,
    this.pengeluaran,
  });

```

```

  @override
  State<FormPengeluaranPage> createState() =>
    _FormPengeluaranPageState();
}

```

```

class _FormPengeluaranPageState
  extends State<FormPengeluaranPage> {
  final _formKey = GlobalKey<FormState>();

  late final TextEditingController _namaController;
  late final TextEditingController _jumlahController;
  late final TextEditingController _tanggalController;

```

```

  String _kategori = 'Makanan';

```

```

  final List<String> _kategoriList = [
    'Makanan',
    'Transportasi',
    'Belanja',
    'Tagihan',
    'Hiburan',
    'Lainnya',
  ];

```

```

  @override
  void initState() {
    super.initState();

    _namaController = TextEditingController(
      text: widget.pengeluaran?.nama ?? "",
    );

```

```
_jumlahController = TextEditingController(  
  text: widget.pengeluaran?.jumlah.toString() ?? "  
);
```

```
_tanggalController = TextEditingController(  
  text: widget.pengeluaran?.tanggal ?? "  
);
```

```
if (widget.pengeluaran != null) {  
  _kategori = widget.pengeluaran!.kategori;  
}  
}
```

```
@override  
void dispose() {  
  _namaController.dispose();  
  _jumlahController.dispose();  
  _tanggalController.dispose();  
  super.dispose();  
}
```

```
Future<void> _pilihTanggal() async {  
  final sekarang = DateTime.now();
```

```
  final tanggal = await showDatePicker(  
    context: context,  
    initialDate: sekarang,  
    firstDate: DateTime(2020),  
    lastDate: DateTime(2100),  
  );
```

```
  if (tanggal == null) return;
```

```
  final bulan =  
    tanggal.month.toString().padLeft(2, '0');
```

```
  final hari =  
    tanggal.day.toString().padLeft(2, '0');
```

```
  _tanggalController.text =  
    '${tanggal.year}-${bulan}-${hari}';  
}
```

```
Future<void> _simpan() async {  
  if (!_formKey.currentState!.validate()) {  
    return;  
  }  
}
```

```

final pengeluaran = Pengeluaran(
  id: widget.pengeluaran?.id,
  nama: _namaController.text.trim(),
  jumlah: int.parse(
    _jumlahController.text.trim(),
  ),
  kategori: _kategori,
  tanggal: _tanggalController.text.trim(),
);

if (widget.pengeluaran == null) {
  await DbHelper.tambah(pengeluaran);
} else {
  await DbHelper.ubah(pengeluaran);
}

if (!mounted) return;

Navigator.pop(context);
}

@override
Widget build(BuildContext context) {
  final baru = widget.pengeluaran == null;

  return Scaffold(
    appBar: AppBar(
      title: Text(
        baru
          ? 'Tambah Pengeluaran'
          : 'Ubah Pengeluaran',
      ),
    ),
    body: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _namaController,
            decoration: const InputDecoration(
              labelText: 'Nama Pengeluaran',
              border: OutlineInputBorder(),
            ),
          ),
          validator: (value) {
            if (value == null ||

```

```

        value.trim().isEmpty) {
      return 'Nama wajib diisi';
    }

    return null;
  },
),
const SizedBox(height: 16),
TextFormField(
  controller: _jumlahController,
  keyboardType:
    TextInputType.number,
  decoration: const InputDecoration(
    labelText: 'Jumlah',
    prefixText: 'Rp ',
    border: OutlineInputBorder(),
  ),
  validator: (value) {
    if (value == null ||
        value.trim().isEmpty) {
      return 'Jumlah wajib diisi';
    }

    final jumlah = int.tryParse(
      value.trim(),
    );

    if (jumlah == null) {
      return 'Jumlah harus berupa angka';
    }

    if (jumlah <= 0) {
      return 'Jumlah harus lebih dari 0';
    }

    return null;
  },
),
const SizedBox(height: 16),
DropdownButtonFormField<String>(
  initialValue: _kategori,
  decoration: const InputDecoration(
    labelText: 'Kategori',
    border: OutlineInputBorder(),
  ),
  items: _kategoriList.map(
    (kategori) {

```



```

        return DropdownMenuItem(
          value: kategori,
          child: Text(kategori),
        );
      },
    ).toList(),
    onChanged: (value) {
      if (value == null) return;

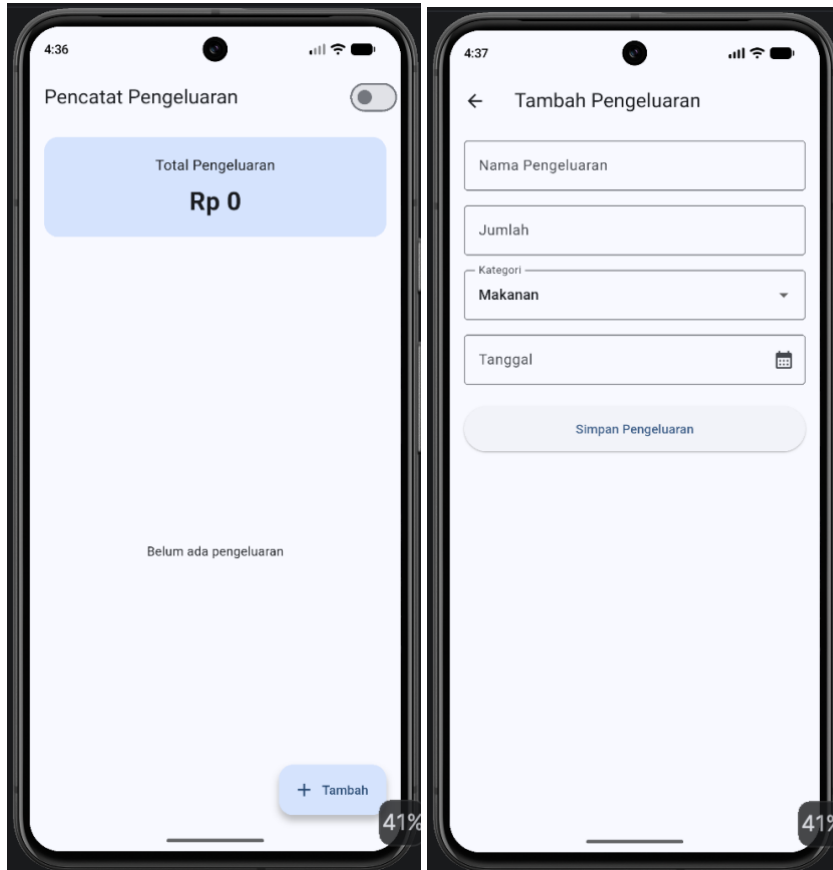
      setState(() {
        _kategori = value;
      });
    },
  ),
  const SizedBox(height: 16),
  TextFormField(
    controller: _tanggalController,
    readOnly: true,
    decoration: InputDecoration(
      labelText: 'Tanggal',
      border: const OutlineInputBorder(),
      suffixIcon: IconButton(
        onPressed: _pilihTanggal,
        icon: const Icon(
          Icons.calendar_month,
        ),
      ),
    ),
  ),
  validator: (value) {
    if (value == null ||
        value.trim().isEmpty) {
      return 'Tanggal wajib diisi';
    }

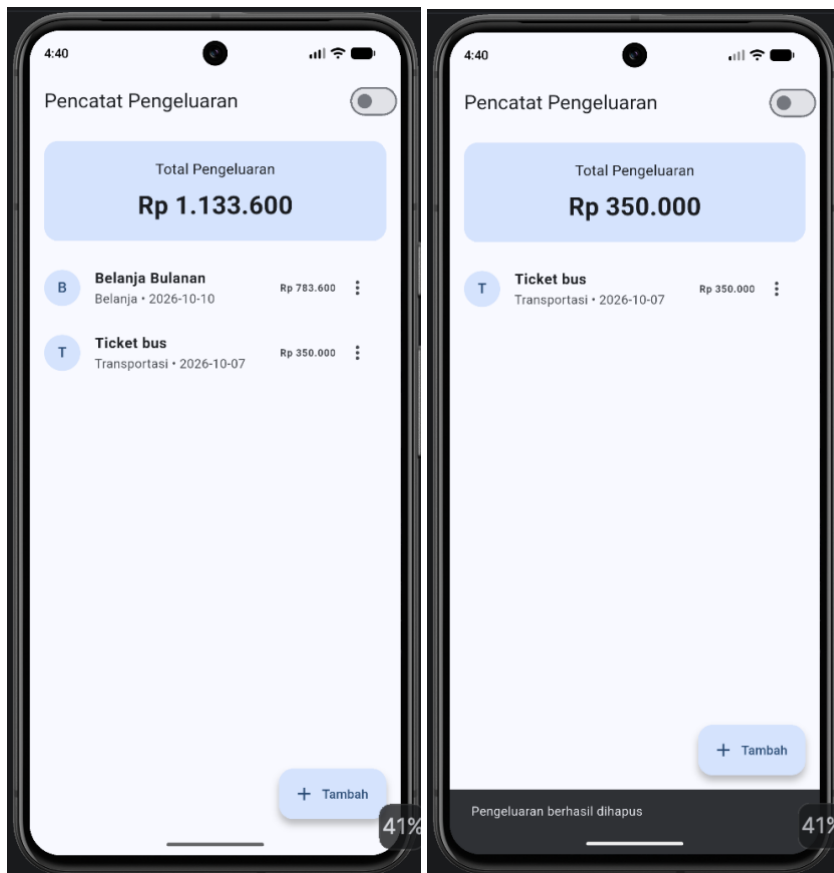
    return null;
  },
),
const SizedBox(height: 24),
SizedBox(
  height: 50,
  child: ElevatedButton(
    onPressed: _simpan,
    child: Text(
      baru
        ? 'Simpan Pengeluaran'
        : 'Simpan Perubahan',
    ),
  ),
),

```

```
),
),
],
),
),
);
}
}
```

Tampilan:





Pertemuan Refleksi

1. Mengapa `shared_preferences` kurang cocok untuk menyimpan ratusan catatan?

Karena `shared_preferences` lebih cocok untuk menyimpan pengaturan sederhana, seperti mode gelap. Untuk menyimpan banyak catatan, lebih baik menggunakan SQLite karena datanya lebih mudah diatur, dicari, dan diubah.

2. Mengapa query memakai placeholder ? dan `whereArgs`?

Supaya data dimasukkan dengan lebih aman dan tidak mudah menyebabkan kesalahan saat mengolah data.

3. Mengapa daftar perlu dimuat ulang setelah kembali dari halaman form?

Karena mungkin ada data baru, data yang diedit, atau data yang dihapus. Pada kode ini, daftar dimuat ulang menggunakan `DbHelper.semua()` setelah kembali dari halaman form.

4. Apa fungsi `onCreate` dan `onUpgrade`?

`onCreate` dijalankan saat database pertama kali dibuat, biasanya untuk membuat tabel. Sementara itu, `onUpgrade` dijalankan saat versi database dinaikkan, misalnya ketika ada perubahan struktur tabel.

